

Energy AI Hackathon 2026 Judging Criteria and Submission Requirements

Official Judging Criteria for Teams (Energy AI Hackathon 2026)

Teams in the 6th Annual Energy AI Hackathon (Jan. 23–25, 2026) are evaluated on a **holistic rubric** covering both the quality of their solution and how well they communicate it. Although the organizers have not published a line-by-line scoring breakdown publicly, the judging framework can be inferred from official materials and past hackathons:

- **Technical Performance & Accuracy:** Solutions are scored on how well they predict the energy usage for the given wells. An automated scoring script compares each team's submitted predictions against the true values (after the event) to compute error metrics. High predictive accuracy (low error) is crucial. In 2025, hackathon organizers set up a CI pipeline to **automatically score submissions** based on the uploaded solution file ¹. This likely contributes significantly to the team's overall score. Teams must include *uncertainty quantification* in their predictions – each well's output requires not only a single point estimate but also 100 probabilistic realizations capturing uncertainty ² ³. (Failing to include the uncertainty component would violate the required format and effectively nullify the submission.)
- **Solution Design & Implementation (Reproducibility and Code Quality):** Judges (and hackathon “architects”) scrutinize the **methodology and code** behind the solution. The hackathon provided a Jupyter Notebook template and expects teams to follow this structure for clarity ⁴. Teams are evaluated on their data science workflow: e.g. how they performed feature engineering, handled missing data, model selection, and hyperparameter tuning ⁵. Good practices like rigorous cross-validation, integration of domain knowledge (energy context), and sound uncertainty modeling earn credit. Code readability, documentation, and reproducibility are explicitly judged – teams must submit their **notebook and any code files** on GitHub, and these are reviewed for clarity and organization ⁴. In short, the solution should be well-engineered and **reproducible by others**. The hackathon architects will review the notebooks for compliance with the template and overall code quality (they even created a `code_review.csv` internally to score this). If the code is sloppy, not well-documented, or cannot reproduce the results, the team will lose points. Each team's workflow should be something “any competent engineer or scientist” could understand and reuse, rather than a black box (this aligns with hackathons emphasizing design and implementation quality, as in typical rubrics focusing on these aspects ⁶).
- **Innovation & Creativity:** Beyond getting a good result, judges look for how **creative or novel** the team's approach is. Did the team just apply a standard model, or did they innovate in feature creation, modeling techniques, or workflow design? Hackathon problems are open-ended, so teams that inject original ideas (for example, clever feature engineering, ensemble models, interactive visualizations, or domain-inspired features) tend to impress the judges. “Innovation” is often a formal scoring category in hackathons ⁶. For instance, using a unique approach to quantify

uncertainty or employing explainable AI techniques to interpret the model would be viewed favorably. The challenge explicitly encourages integrating domain expertise at every step ⁵ – effectively rewarding teams that combine oil & gas domain knowledge with AI in creative ways. Thus, a solution that is not only accurate but also insightful (e.g., revealing which factors drive energy usage, or providing decision-support visuals) can score higher on creativity and real-world usefulness.

- **Presentation & Communication:** The hackathon culminates in team presentations to a panel of industry experts on Sunday (Jan. 25) ⁷ ⁸. **How well the team communicates their problem understanding, approach, and results** is a major part of the judging. Teams must distill their complex analysis into a clear, compelling story for the judges. This includes the quality of slides, visualizations of results, speaking clarity, and the ability to answer questions. In fact, having a polished and informative presentation can be a **deciding factor** – for example, in other AI hackathons, the presentation's clarity and narrative carried significant weight in scoring ⁹. Judges will be asking: *Does the team explain their model and findings clearly? Did they visualize the predictions and uncertainty in an understandable way?* Expect that **data visualization** and interpretation are implicitly rewarded; teams often include plots of actual vs. predicted values, feature importance rankings, uncertainty interval illustrations, etc., to strengthen their story. The hackathon even provided a PowerPoint template to guide participants in crafting their story ¹⁰, underlining how crucial a well-structured presentation is. In short, a technically strong solution needs an equally strong presentation to score well.
- **Potential Impact and Completeness:** While this competition is focused on a specific problem (energy usage forecasting), judges may consider the broader impact or completeness of the solution. For example, does the team's workflow have potential for real-world implementation or future extension? Did they address all parts of the task (all fuel types, uncertainty, etc.) comprehensively? Hackathon judges often informally reward teams that go **above and beyond** – such as delivering a solution that could be used by industry or that shows clear thinking about “what's next”. In some hackathons, this is framed as “Future Potential” ⁶ or simply reflected in the overall impression score. In the Energy AI Hackathon, this could translate to considering how robust or scalable the solution is for different datasets or whether the team identified limitations and next steps. It's not a strictly defined category, but teams that demonstrate a broader vision (for example, how to deploy the model or how it could save costs/environmental impact) could earn bonus kudos from judges.
- **Adherence to Guidelines:** While not a point-scoring category per se, it's critical to follow all the hackathon's format and submission rules. Any **disqualifying mistakes** can nullify an otherwise great project. For instance, the **submission format requirements are strict**: the solution CSV must have exactly the provided columns and rows (including the 100 `Real_#` uncertainty columns) with no alterations ¹¹. The organizers warned that any changes to file format (e.g. renaming columns, adding extra rows) “**will prevent your solution from being scored**” ¹¹. Similarly, failing to include the uncertainty realizations or other required outputs would make the submission incomplete. Teams also had to meet team composition rules (each team needed at least one UT PGE student and one undergraduate student) ¹², though this is checked at registration rather than judged for points. In summary, teams are expected to **strictly comply** with all instructions: correct file naming, format, using the provided GitHub repo for submission, and deadlines. Compliance is essentially the

ticket to entry for judging – if you slip up on the format or miss the noon deadline, your solution might be disqualified before it's ever scored.

Summary of Judging: In practical terms, each team's performance will be measured across multiple dimensions. The hackathon's scoring process likely combines an **automated score** for prediction accuracy with **qualitative scores** from judges for categories like technical rigor, creativity, and presentation quality. This is analogous to many hackathons that score teams on Innovation, Technical Implementation, and Presentation on a 1–10 scale ⁶ ¹³. Therefore, to win, a team must not only build an accurate model but also demonstrate solid engineering practices, originality, and the ability to communicate and justify their solution. A well-rounded effort covering all these bases will fulfill the “complete judging criteria” that the Energy AI Hackathon 2026 is looking for.

Submission Requirements and Deliverables

The official hackathon instructions specify several **deliverables** that each team must submit by the end of the competition, along with formatting rules. The submissions are all handled via a GitHub repository provided to the team (the PGE Hackathon GitHub organization), and the deadline for final commits is typically noon on the day after the hackathon (for 2025 it was Jan. 26 at 12:00pm, likely similar for 2026) ¹⁴.

Participants must prepare the following:

1. **Prediction Output CSV** (`solution.csv`): This file contains the team's final energy usage predictions and uncertainty quantifications for the 50 test wells. The hackathon provided a template `solution.csv` file, and teams **must follow this exact format** ¹⁵ ¹¹. Each row corresponds to a well and a fuel type, identified by `Well_ID` and `Fuel_Type` (note: for dual-fuel DGB wells, there will be two rows – one for Diesel, one for CNG – so the CSV has 63 rows for 50 wells) ¹⁶ ¹⁷. The columns include a single **“Point Estimate”** (the team's best guess for energy usage in that row's units) and **100 columns of uncertainty realizations** (`Real_1` through `Real_100`) representing the distribution of possible outcomes ¹⁸. **No deviation is allowed** in this file: teams must *replace the placeholder values* (e.g., -999 or NaNs) in the template with their predictions, but **cannot alter the column names, add extra columns, or change the number of rows** ¹¹. The file must be named exactly `solution.csv` ¹⁹ and committed to the repo for automated scoring. A submission that does not include 100 uncertainty realizations for each prediction (or breaks the format in any way) will not be scored by the system ¹¹. This requirement enforced that teams implement some form of uncertainty estimation in their workflows. *(In 2025, for example, teams used methods like residual bootstrapping to generate the required 100 realizations* ²⁰.)
2. **Jupyter Notebook Workflow** (`<TeamName>.ipynb`): The core of the technical submission is a Jupyter Notebook containing the team's data science workflow – from data ingestion and cleaning through feature engineering, modeling, and generating the final predictions. The hackathon organizers provided a **notebook template** (“Hackathon_ProjectTemplate.ipynb”) as a guide, and they require teams to follow this structure for consistency and readability ⁴. The submitted notebook should be **well-documented** with markdown text, explaining the approach and rationale, and should include any relevant analysis (plots, tables) that support the solution. Essentially, this notebook should allow the judges and hackathon architects to **understand exactly what the team did** and to rerun the code to reproduce the results. All custom code (if not in the notebook) and any additional data files needed to run it must also be included in the repository ⁴ – for example, if the team created helper modules or used additional datasets, these need to be present so that there are

no missing dependencies. The notebook file must be named after the team (the instructions say “the file must be named `xxxx.ipynb` where `xxxx` is your team name”) ²¹. This naming convention helps organizers identify each team’s submission. The notebook is submitted by committing it to the team’s GitHub repo by the deadline ¹⁴. Hackathon staff will review this notebook for judging the code quality/reproducibility aspects, and it may also be shared with industry judges to reference technical details if needed. A clean, well-structured notebook following the template is crucial for maximizing the score on the “technical implementation” and “reproducibility” criteria.

3. **Presentation Slide Deck** (`<TeamName>.pptx`): Each team must also submit a PowerPoint presentation that will be used for their final pitch to the judging panel ¹⁰. A **presentation template** (“Hackathon_PresentationTemplate.pptx”) was provided to teams as an example format to follow. The expectation is that the deck will contain an overview of the problem, the team’s approach (methodology), key results (with visuals), and conclusions/impact. Essentially, it’s the storytelling vehicle for the team’s project. The **first slide must include the project title and the names of all team members** ²² – failing to list team members was explicitly called out as something to avoid. The file should be named `<TeamName>.pptx` (just like the notebook) and pushed to the GitHub repo for the judges to access ahead of the presentations ¹⁰. Judges will have these files when evaluating the talks, and they might refer to them. It’s important that the slides are clear, not overly packed, and effectively communicate the findings. Teams can include additional slides (e.g., backup slides with extra charts or code snippets) if they anticipate technical questions. While the live presentation is what the judges see, this submitted deck is a record of the content and is also reviewed. Thus, teams should treat the slide deck as a required deliverable on par with the code. The hackathon rules emphasize using the provided template and including all names, again highlighting the importance of a professional and compliant presentation file ²².

4. **(Optional/Supporting) README or Documentation**: Although not explicitly listed as a required deliverable in the hackathon instructions, many teams include a README.md in the repository (and the provided repo template includes one ²³). The README usually gives a high-level description of the solution, how to run the code, and any other notes. In 2025, the template README outlined the problem statement and available data ²⁴ ²⁵. For 2026, teams might update this with their specific approach. While judges primarily focus on the three main items above, having a clear README is good practice and can help anyone browsing the repo (including recruiters or future team members).

Submission Process: All these deliverables are to be submitted through the team’s GitHub repository on the hackathon’s organization. Teams are expected to commit their final code, data, and files, then push by the deadline ¹⁴. After the deadline, an automated system pulls the submissions for scoring. For example, once `solution.csv` is pushed, the CI scripts will run to calculate the team’s score on the prediction task ¹. Meanwhile, hackathon staff and judges will download the notebooks and presentations for review. It’s each team’s responsibility to double-check that their files are correctly named and uploaded. Missing any of these required files (or having naming mismatches) could result in a team being **disqualified or not scored**. The organizers made it clear that *formatting issues* can have severe consequences: e.g., if you alter the solution file format even slightly, the automated scorer might skip your team ¹¹. In short, **meeting the submission specs to the letter is as important as the solution itself** – a great project that isn’t packaged correctly for submission won’t win.

Finally, note that **timeliness** is critical. The hackathon timeline (Jan 23–25, 2026) indicates presentations are on Sunday the 25th, and in 2025 final files were due by Monday noon ¹⁴. Teams should plan to finish coding and slide preparation in time to submit comfortably before the cutoff. Late submissions likely won't be accepted (or would incur penalties). Part of the hackathon challenge is delivering under time pressure, so managing version control and using Git well (which the team in question has done via Replit/GitHub integration ²⁶) is key to avoiding last-minute errors.

Gap Analysis: Current Project vs. Hackathon Requirements

Now, let's compare the user's current project state (as described in `PROJECT_CONTEXT.md` and the provided workflow files) against the official criteria and requirements above. The goal is to identify what is **already strong or complete** and what is **missing/incomplete** in terms of being fully "judge-ready."

Strengths / Completed Elements: The team's current workflow has several positives that align well with hackathon expectations:

- **Correct Prediction Format Implemented:** The project already produces a `solution.csv` with the required columns and structure for the hackathon. The context file confirms that the solution file contains 63 rows (accounting for DGB wells) and includes columns for `Well ID`, `Fuel Type`, a point estimate, and `Real_1` through `Real_100` uncertainty values ¹⁸ ¹⁶. This matches the exact format required by the competition. Having this in place is a major compliance item. The team also explicitly used **residual bootstrapping** to generate 100 realizations, noting that this was *"required by hackathon submission format"* ²⁰. This shows they understood and met the uncertainty quantification requirement. No gaps are seen here; just ensure to regenerate this CSV with the **2026 test data** once available.
- **Robust Technical Approach:** The team made sound modeling decisions that address the problem requirements. They chose to train separate Random Forest models for each fuel type (Grid, Diesel, CNG, and dual-fuel DGB) ²⁷ ²⁸, which avoids predicting irrelevant zeros and respects the domain logic (since DGB wells need two outputs). This specialization strategy is reasonable and was justified in their context as improving accuracy ²⁹. They also performed cross-validation to estimate performance, which is good practice for robustness ³⁰. The workflow includes steps for feature engineering (`src/features.py`) and modeling (`src/modeling.py`), indicating a modular and reusable code structure ³¹ ³². The use of residual bootstrapping for uncertainty provides a realistic spread of predictions ³³. All these technical decisions align with hackathon expectations (e.g., they *did* integrate a form of domain knowledge by handling fuel types separately, which fulfills the "integration of domain expertise" spirit ²⁷). So technically, the solution appears solid and likely to perform reasonably well.
- **Reproducibility and Environment:** The project context outlines a consistent development environment (Python 3.11, specific packages listed) and even notes the use of a Replit workspace and GitHub repo for version control ³⁴ ²⁶. This is a good indicator that the team can **reproduce their results** and that they have a controlled environment – important for judges who might try to run the notebook. They have a `requirements.txt` and documented the directory structure of the project ³⁵ ³¹, which is exactly what one would expect in a well-organized submission. The fact that the project is set up for "plug-and-play" with new data (the context explicitly lists steps to adapt to

the 2026 hackathon: replace data files, update features, run notebook) ³⁶ shows foresight. In principle, their repository is already structured very similarly to the official template (since it contains the `src/`, `notebooks/`, `outputs/` folders, etc. that mirror the hackathon's template repo structure ³¹ ³⁷). This means they are largely in compliance on the code organization front. No major issues here – just ensure that all necessary files (like any data or custom modules) are indeed in the repo when submitting.

- **Current Progress Status:** As of the context's last update (Current State: January 2026), the team believes they are *“ready for submission or continued refinement”* ³⁸. All models are trained and validated, the solution file is generated, and uncertainty realizations are computed ³⁸. In other words, the team is not scrambling to get things working – they have a complete pipeline that yields a result. This is a great position to be in before the hackathon final stretch, because it means the basics are done. The gap analysis below will focus on **enhancing and packaging** this work to maximize scores, rather than fixing any fundamental flaw.

Despite these strengths, there are several **gaps and to-dos** to address to achieve full compliance with the judging rubric and submission requirements:

- **Documentation and Narrative in the Notebook:** The current main workflow notebook (`main_workflow.ipynb`) may not yet be in the polished form that judges expect. The hackathon template likely expects sections like *Introduction*, *Data Understanding*, *Feature Engineering*, *Model Training*, *Results*, *Conclusion*, etc., with **explanatory markdown** in each. It's not clear if the team has already added extensive markdown explanations or if the notebook is mostly code. Given the focus on code readability ⁴, this is a gap to check. To align with criteria, the notebook should tell a story: why the team chose Random Forest, how they handled each fuel type, how they ensured no data leakage, how uncertainty was computed, and some analysis of results (e.g., feature importance or error analysis on training data). If the current notebook lacks these discussions, the team needs to incorporate them. Essentially, the notebook should read like a mini-report. This will directly impact the “technical implementation” and possibly “innovation” scores, because well-justified decisions make the solution look more thoughtful.
- **Alignment with Notebook Template & Naming:** Related to the above, the submitted notebook needs to **follow the hackathon's template format** closely ⁴. The project context mentions the existence of `Hackathon_ProjectTemplate.ipynb`, but it's unclear if the team's current workflow is written *inside* that template or if they built a separate notebook. If not already done, the team should merge their content into the template structure (or at least mirror its section headings). For example, the template might have had placeholder sections for loading data, EDA, methodology, etc. The judges will be familiar with the template, so deviating from it might make the notebook harder to read. Moreover, the **filename of the notebook** must be changed to the team name before submission ²¹. Currently, it's named generically (`main_workflow.ipynb`). Forgetting to rename it “TeamX.ipynb” would be a compliance miss (the instructions were explicit on this). This is a simple but important gap to fill. The team should decide on a team name if not already, and ensure all submission files use that naming convention.
- **Presentation Deck Content:** The context lists a `Hackathon_PresentationTemplate.pptx` in the repository, but presumably this is still the blank template or an outline. The **actual presentation needs to be created** using this template. This is a significant piece of work remaining. The team

needs to condense their project into a clear slide deck, typically ~5-10 slides for a hackathon presentation (depending on the time limit, often ~10 minutes per team). Key gaps likely include: formulating an introduction that frames the problem, selecting the most important results to showcase (e.g., maybe a chart of predicted vs actual energy usage for some wells, or a bar chart of feature importances), and practicing how to explain the uncertainty quantification to a general audience. They should also prepare to answer judges' questions about why they chose certain methods. At minimum, **all team members' names must be on the title slide** ²² – the team should do this first to avoid forgetting. If the current deck has no content, this is a major to-do. If it has content, it should be reviewed for clarity and adherence to the template styling. Remember, presentation quality will be a **major scoring factor** as noted earlier, so any weakness in storytelling or visual communication is a gap to close.

- **Visualizations and Exploratory Analysis:** From the project context, it's not obvious how many data visualizations or exploratory analyses the team has done. The only mention is that an `images/` directory exists (possibly for plots). Judges appreciate seeing that the team **understood the data** through visualizations (for example, distributions of energy usage, correlations between features and target, etc.). If the current workflow lacks visuals like graphs or charts (either in the notebook or slides), the team should add a few. For instance, showing a plot of actual vs predicted values for the training set (with uncertainty bounds) could demonstrate how well the model fits. Or a feature importance chart from the Random Forest could highlight which predictors mattered most – adding interpretability, which is often rewarded. Also, consider including any insights discovered (e.g., “we noticed that wells in Formation X consistently needed more diesel, which the model captured”). These kinds of domain-relevant insights make the project more compelling and show creativity/understanding. If the team hasn't done this analysis due to time, it's a missed opportunity. It's not strictly required to predict values, but it **boosts the presentation and innovation scores** if done. This gap is moderate – not a compliance issue, but a quality issue.
- **Domain Expertise Integration:** The hackathon problem statement emphasizes using domain knowledge (oil & gas context) in the solution ⁵. The team's approach of splitting by fuel type is indeed a domain-driven choice (since it respects how different frac fleet setups consume different fuels). That's good. Are there other places where domain insights could be injected? For example, maybe certain features like “# of Stages” or “Target Formation” have known impacts on fuel consumption – did the team consider interactions or non-linear relationships that a domain expert might expect? If not, they might mention in their discussion that domain expectations were considered (even if the model ultimately sorts it out). Another domain aspect: understanding the cost or logistical implications of over/under-predicting energy. If the team can articulate why their predictions and uncertainties would help an operator make better decisions (e.g., “with our uncertainty range, planners can ensure with 95% confidence that they allocate enough diesel for each job, avoiding costly shortages or oversupply”), that ties the project back to real-world impact. In the current materials, such reasoning isn't explicitly documented, but adding a sentence or two either in the notebook conclusion or in the presentation about “*what this means for industry*” can impress judges. This is a minor gap, but attending to it can move the project from a purely academic exercise to a practical solution – often a tiebreaker in judging.
- **Code Cleanup and Repository Polish:** The team's codebase should be checked for any last-mile issues. For example: Are there any hard-coded file paths that might break when the repository is cloned fresh? Is every dependency listed in `requirements.txt`? Since they used Replit, did they

include any environment-specific configurations that need removal (e.g., in `replit.md` or elsewhere)? The context does list a `replit.md` and other config files ³⁹ ⁴⁰. Those are fine to include, but the team should be mindful that judges will see everything in the repo. It should be tidy and professional. That means removing any debugging printouts or prototype code, and making sure the README is up-to-date (if it mentions 2025, update it to 2026 context to avoid confusion). Also, ensure that the notebook runs from start to finish without errors on a clean environment. A good practice would be to do a fresh clone of the repo (or ask a team member who hasn't run it recently) to run the notebook from zero and see if any issues arise. If, for example, large data files are needed, ensure they are in the repo or clearly instruct how to get them. The project context suggests the main data files are included (`HackathonData2025.csv`, etc.) ⁴¹; for 2026, the new data will have to be added similarly. This polish is often what distinguishes a *fully compliant, easy-to-evaluate submission* from a messy one. It can indirectly affect scores (especially code review scores). Currently, no glaring gap is described here, but it's a checklist item to verify.

- ● **Testing on 2026 Data:** While the team could not have the 2026 challenge data until the event, their plan is to simply swap in new CSVs and rerun the pipeline ³⁶. It's worth considering any changes that might come with the new data. For instance, if the 2026 data has new features or slightly different formatting, is the code robust enough to handle it? The context mentions updating feature lists if columns change ⁴², so they are aware of this. The gap here is to ensure that process goes smoothly during the hackathon. The team should allocate time to do this swap and test early in the competition window. It would be wise to generate a quick dummy "testing.csv" with a different pattern to simulate and make sure the `generate_solution` function still produces correct output. Essentially, the **gap is uncertainty about future data** – which can only be solved by being prepared. Since this isn't something the judges see directly, it's more about risk management for the team. But failing to adapt to the new data in time would of course ruin the submission. Given their preparedness, this is low risk, but keep it on the radar.
- ● **Compliance with File Naming & Team Info:** Double-check that all files follow naming rules: `solution.csv` (already correct), `TeamName.ipynb`, `TeamName.pptx`. Also check that the first slide of the PPT has everyone's name (the rules specifically mandate this ²²). It's easy to overlook a teammate's name or use an acronym for the team name that isn't the same as what you registered – ideally the team name on slides should exactly match what was given to organizers to avoid confusion. If the team has not yet finalized a team name, that's an immediate task. In 2025, team names were often creative but still professional. For now, this is a gap only in the sense that it's a detail that could be missed in a rush. It's straightforward to fix: pick a name and apply it consistently.
- ⚠ **Potential Format Pitfalls:** While the team's solution format is correct now, they should be careful when regenerating it for submission. A common mistake could be, say, leaving index columns from pandas in the CSV or changing a header by mistake. Also, using floating-point numbers can introduce formatting issues (scientific notation, too many decimals, etc.). The template likely expects a certain numeric precision. The team might want to format their output numbers to a reasonable number of decimal places for presentation – e.g., `%.2f` – unless instructed otherwise. The context doesn't mention if they handle that. Slight inconsistencies (like "Well ID" vs "Well_ID") would break format compliance. Given they likely started from the official `solution.csv` template, they should be fine. Just treat the template as gospel when producing the final output. This is more a reminder than a gap, but worth highlighting: **in final submission, no format errors can be afforded** ¹¹.

In summary, the team's project is **technically on track**, but to be fully *judge-ready*, they need to focus on **presentation and polish** in the remaining time. The gaps identified – chiefly around documentation, presentation slides, and compliance details – are all about how the work is delivered to the judges. None of these require fundamentally new technical development (which is good news at this stage), but they will decide whether the team's hard work translates into a winning impression. The next section provides specific recommendations to close these gaps.

Recommendations to Achieve Full Compliance and Maximum Score

To ensure the project meets all judging criteria and submission requirements, the team should take the following actions:

- 1. Incorporate the Notebook Template and Polish Documentation:** Immediately integrate your code and analysis into the official hackathon notebook format if you haven't already. Use clear section headings (following those in `Hackathon_ProjectTemplate.ipynb`) and add **explanatory markdown** for each step of your workflow. Begin the notebook with a brief introduction to the problem and your approach. In the modeling section, write a few sentences on why you chose Random Forest and separate fuel models (e.g., *"to avoid predicting zero for unused fuels, we modeled each fuel type independently, which improved accuracy"* ²⁹). When presenting results, include at least one plot or table to illustrate performance (for example, a small table of cross-validation errors for each fuel model, or a graph of predictions vs actuals for a couple of sample wells). End the notebook with a conclusion that reiterates the accuracy achieved and notes any interesting findings (e.g., which features were important). This level of narration will satisfy the hackathon architects' expectations for a well-documented workflow ⁴ and help judges follow your reasoning. Also, verify that your notebook runs from start to finish without errors in a fresh environment. This means clearing all outputs and re-running to catch any hidden dependencies or ordering issues. A fully polished notebook not only scores better on reproducibility and technical merit, but also makes it easier for you to pull content for your presentation.
- 2. Finalize File Naming and Include Team Details:** Decide on your official team name (if you haven't already from registration) and rename your files accordingly. Your Jupyter notebook should be renamed to `<TeamName>.ipynb` and your presentation to `<TeamName>.pptx` ²¹. For example, if your team name is "PowerPredictors", your files would be `PowerPredictors.ipynb` and `PowerPredictors.pptx`. Update any internal references if needed (e.g., if your README still says "Hackathon 2025", change it to 2026 and include your team name there too for consistency). **On the title slide of your PowerPoint, list every team member's full name and affiliation** (e.g., "Alice (UT PGE), Bob (ECE), ..."). The rules mandate having all names on the presentation ²², and judges will note if someone is missing. This is an easy place to avoid a penalty – double-check spelling and inclusion of each person. It's also a courtesy to acknowledge everyone's contribution. Commit and push these renamed files early to ensure the organizers see them and there's no confusion mapping your submission to your team.
- 3. Develop a Clear and Compelling Presentation:** Allocate significant time to build your slide deck, as it carries considerable weight in judging ⁹. Start by outlining the story: *Problem* → *Solution Approach* → *Results* → *Business Impact*. Aim for ~8–10 slides:

4. **Title Slide:** Project name, team name, team members (with university/department if you want) ²² .
5. **Problem Statement:** One slide summarizing the task (e.g., “Predicting energy usage (Grid, Diesel, CNG) for hydraulic fracturing operations, with uncertainty, to optimize power planning” – essentially draw from the hackathon prompt). Emphasize why this is important (cost, efficiency, etc.), in layman’s terms.
6. **Dataset & Challenges:** One slide briefly describing the data you had (number of wells, features, any notable missing data or outliers). If there were domain challenges (like highly variable usage, or DGB dual fuel handling), mention those to set context for your approach.
7. **Approach/Methodology:** 2–3 slides. Explain that you trained separate models per fuel type ²⁷ and why. A small schematic or flowchart could help (e.g., a diagram with wells split into 4 groups feeding into 4 Random Forest models, then outputs recombined into the solution.csv format). Bullet points can outline key techniques: *“Random Forest Regressor with cross-validation (robust to overfitting)”* ³⁰ ; *residual bootstrapping to quantify prediction uncertainty (100 realizations)”* ²⁰ ; *custom feature engineering (mention any important ones); etc.* Keep text minimal – use diagrams or images if possible (maybe an image of a rig or fuel types, just for visual interest, or the illustration provided in the data description ⁴³).
8. **Results:** 1–2 slides. Present your model’s performance. Since you don’t have ground truth for test wells (that’s hidden for scoring), you can show your cross-validation or training performance as a proxy. For instance, “Achieved X% R^2 or Y RMSE on validation.” If you can, show a plot: perhaps a bar chart comparing actual vs predicted energy for a few sample wells (with error bars showing uncertainty). Or a graph of prediction intervals capturing the actual value. Also, report that you met the format: e.g., “Generated solution.csv with 100 uncertainty realizations per well” – showing you accomplished the core requirement. If you discovered anything interesting (e.g., *“Diesel usage was generally harder to predict than grid electricity – higher variance in residuals”*), this is a good place to mention it. Ensure any figures are readable (proper labels, not too small).
9. **Uncertainty Interpretation:** It might be worth dedicating a small portion of results to explain what the uncertainty means in practice. For example, *“Our 100 realizations per well can be used to construct a prediction interval – roughly, we expect the true usage to fall in the range [min, max of those realizations] for each fuel. On validation data, 90% of true values fell within our predicted band, indicating our uncertainty quantification is well-calibrated.”* Explaining this in non-technical terms can impress judges (it shows you understand the value of uncertainty, not just doing it because it was required).
10. **Conclusion/Impact:** 1 slide. Summarize how your solution can help. For instance, *“With our model, an operator can predict how much diesel and CNG will be needed for a job ahead of time, with a quantified uncertainty of $\pm 10\%$. This helps avoid under-supply (preventing job delays) and over-supply (avoiding wasted fuel costs).”* Even if rough, giving a real-world implication (maybe mention potential cost savings or efficiency gains) will leave a strong final impression. Also, if appropriate, mention potential next steps or improvements (shows you thought beyond the competition), but keep it brief.
11. **Questions Slide:** (optional) Have a final slide ready in case of Q&A, maybe with backup info like a feature importance plot or any technical detail you expect to be asked about (e.g., hyperparameters of the Random Forest).

Once the slides are made, **practice your delivery** as a team. Time it to make sure it fits the allotted time (often ~10 minutes). Ensure each member speaks (most hackathon judges like to see team collaboration). Focus on being clear and avoiding jargon when simpler terms suffice (remember, not all judges may be ML experts – some might be energy domain experts looking for insight). Given that presentation was highlighted as a major factor in scoring, put as much effort here as you did in coding the model ⁹ . A tip: use visuals where possible (graphs, icons) instead of text bullets, to make the presentation engaging. If you have time and the ability, you could even include a short **interactive element or demo**, but since this

hackathon is about a static prediction task, an “interactive visual” could simply mean a well-animated plot or a brief live demonstration of filtering through your results. This is not strictly required, but going a little beyond with presentation creativity (as long as it’s relevant) can make you memorable.

1. **Double-Check the Solution CSV Before Submission:** When you integrate the new 2026 test data and generate the final `solution.csv`, take a minute to verify it against the template one more time. Manually check that:
2. The **column names** exactly match (`Well_ID`, `Fuel_Type`, `Estimated Energy Use`, `R_1`, `R_2`, ... `R_100`) or whatever the template uses – note whether there’s a space or underscore in “Well ID” etc., and replicate it precisely ¹⁵.
3. The number of rows is correct (if 50 wells with some DGB, should be 63 rows as in 2025). If the 2026 test set has a different composition of fuel types, ensure your code outputs the appropriate rows for each (your design of separate models per fuel type should handle this as long as you loop through all wells).
4. All placeholder values are replaced – no “-999” or NaNs left anywhere ¹¹.
5. Numeric formatting is reasonable: typically, two to three decimal places are fine. Extremely long floats or scientific notation might not look good; you can format the output if needed. (The scoring program likely can parse either, but the safer and cleaner, the better.)
6. The file is named `solution.csv` (all lowercase). No prefix, no team name in it, and placed in the root of the repository or specified location as required. The instructions explicitly say the file must be named `solution.csv` ¹⁹.
7. After generating it, **open the CSV in a text editor or Excel** to visually inspect it. This can catch any encoding issues or an extra delimiter at the end of lines, etc. Also ensure you didn’t accidentally include an index column as the first column (if using pandas `to_csv`, remember to set `index=False`).

Because any format mistake will prevent scoring ¹¹, it’s worth the extra few minutes to do this QA. Since you already succeeded with the 2025 data, you’re familiar with the format – just don’t get complacent with the new data. If possible, have one team member solely responsible for validating the submission files while others are preparing the presentation, as a checks-and-balances approach.

1. **Enhance Interpretation and Explainability:** To address the “innovation/creativity” aspect and also strengthen your presentation’s Q&A, consider adding some explainability analysis. For example, you could compute feature importances from your Random Forest models (scikit-learn provides `.feature_importances_`). If you have time, do this for each fuel model and see which features are driving predictions. You might find, say, for Grid energy, “# of Clusters” is very important, whereas for Diesel, “Ambient Temperature” matters more (hypothetically). These insights are gold for impressing judges – it shows you didn’t treat the model as a black box; you tried to learn from it. You can mention these in the presentation (“Our model indicates that the number of stages and the target formation are the top predictors of diesel usage, which makes sense geologically because...”). This ties into demonstrating domain understanding and the why behind your model’s behavior. If you have even more time, plotting partial dependence or SHAP values for one model could be next-level, but only do that if you’re comfortable with those tools and can concisely explain them. It’s not worth confusing judges with overly technical graphs. The goal is simply to show **you understand what your model is doing**, and thus can trust its predictions. This will give judges confidence in your solution’s viability. Given that explainability wasn’t explicitly required, it counts as a bonus – it can differentiate you from other teams that might just present predictions with no interpretation. In

terms of gap: if your current workflow has none of this, it's a nice improvement to make. Just be mindful of time; prioritize core deliverables first (format, basic slides) then add this if time permits.

2. **Validate Reproducibility with a Fresh Run:** Before the final submission, do a final "dry run": clone your GitHub repo into a new directory (or ask a teammate who hasn't been running the code on their machine) and execute the notebook from scratch, using the instructions you plan to provide. This simulates what judges/organizers will do when they review your code. Because you've documented the environment (Python 3.11 and required libraries) ³⁴, try to use that environment. This test can catch things like missing package imports, or data files not pushed to GitHub. For instance, ensure that the large CSV data (training and test) are indeed in the repo or accessible. In 2025, those files were part of the provided template ⁴⁴ ⁴⁵, so likely 2026 will too – but if you had to pull data from elsewhere (e.g., a private link), make sure that step is either not needed or clearly documented. Given you've included everything in `data/`, you should be fine. Also test the generation of the solution file in this dry run and compare it to your earlier one for consistency. Essentially, you want to verify that someone following your "How to Run" instructions ⁴⁶ will end up with the same `solution.csv`. If the runs are deterministic (they should be, unless bootstrapping adds randomness – if so, set a random seed for consistency), then you're in good shape. Document any non-deterministic aspects (e.g., "realizations may vary slightly each run, but that's expected"). This level of diligence in reproducibility will shine through in judging. The hackathon architects might even run an automated script to execute all team notebooks – you want yours to be one that runs without a hitch.

3. **Time Management and Final Checklist:** On the last day of the hackathon, use a checklist to ensure you haven't missed anything:

4. All required files (`solution.csv`, notebook, PPT) are present and up-to-date in the repo.
5. Files are correctly named and team name is consistent everywhere ²¹.
6. The solution CSV has been double-checked for format ¹¹.
7. The notebook has clear markdown explanations and has been run fully.
8. The presentation is polished, and you've rehearsed the delivery as a team.
9. You've included team member names on slides and in the repo README or context (credit where due).
10. You have pushed the final commits to GitHub **before the deadline** and verified that the files are visible on GitHub. (It's wise to do this at least an hour early to buffer against any Git or internet issues. Remember the deadline is hard – in 2025 it was noon Monday ¹⁴, likely similar for 2026. Don't be the team uploading at 12:05 PM.)
11. (Optional) If the organizers provided a submission checklist or if there's an online portal (less likely here since GitHub is the primary), make sure to complete any form or email that goes with the submission.

By following these recommendations, the team will address the current gaps and align their project with **all judging criteria and submission requirements**. In effect, you'll turn a technically sound project into a fully compliant, presentation-ready package. The result should be a submission that **scores highly across the board** – technically correct, creatively insightful, well-documented, and convincingly presented. Good luck at the hackathon, and congratulations on getting this far with a strong project!

Sources: The above guidance is based on the official Energy AI Hackathon 2025/2026 instructions and templates (University of Texas at Austin), as well as general hackathon best practices. Key references include the hackathon README and data repo ^{2 15 47}, which detail the required deliverables and format, and the judging considerations inferred from those rules and prior events ^{20 9}. All specific requirements quoted (file formats, naming, deadlines, etc.) come from these official materials ^{11 21}. The recommendations here aim to ensure full compliance with these guidelines and to maximize the team's scoring potential in each judging category.

¹ GitHub - PGEHackathon/scoring

<https://github.com/PGEHackathon/scoring>

^{2 3 4 5 10 11 14 15 19 21 22 23 24 25 43 44 45 47} GitHub - PGEHackathon/data2025

<https://github.com/PGEHackathon/data2025>

^{6 13} Hackathon Judging Criteria | PDF

<https://www.scribd.com/document/857585011/Hackathon-Judging-Criteria>

^{7 8 12} Energy AI Hackathon - Hildebrand Department of Petroleum and Geosystems Engineering | The University of Texas at Austin

<https://www.pge.utexas.edu/energy-ai-hackathon/>

⁹ PG&E shows how comms pros can turn everyday skills into practical AI wins - Ragan Communications

<https://www.ragan.com/pge-ai-practical-wins/>

^{16 17 18 20 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 46} PROJECT_CONTEXT.md

file:///file_00000000e064722fbd6423c8ffc8d8ed